

VIEDIT — a vi-style editor for CP/M 2.2

VIEDIT is a small, modal, vi-style full-screen text editor for CP/M 2.2, written in Intel 8080 assembly and built with Microsoft M80/L80. It targets a VT100-compatible terminal and runs on any CP/M 2.2 system with enough TPA.

This file is a high-level overview. The full end-user manual is [VIEDIT.md](#).

Overview

- **Modal.** Like vi, the same keys mean different things in **command mode** (the default — keystrokes move and edit) and **insert mode** (keystrokes are text). `ESC` always returns to command mode. The status line shows `-- INSERT --` / `-- REPLACE --` when applicable.
 - **Edits in RAM, pages when it must.** A file that fits the TPA is held resident, so every operation is fast (no paging to disk). A file too large to fit switches automatically to a paged **virtual mode** — a window slides over the document while the rest lives in disk scratch — so size is no longer a hard ceiling, at some cost in speed and a few restrictions (see *Limitations*).
 - **Byte-for-byte preservation.** What you open is what you save, apart from your edits — lone `CR`s, bare `LF`s, and a missing final newline are all kept. New files use standard CP/M `CR`, `LF` lines with a trailing `^Z` pad.
 - **Terminal aware.** Uses VT100 escape sequences for screen control and can auto-detect the terminal size at startup (VT100 cursor-position report).
-

Running it

```
VIEDIT [filename] [/Ln] [/Cn] [/R] [/B]
```

Argument	Meaning
<code>filename</code>	File to edit. Omit to start with an empty, unnamed buffer.
<code>/Ln</code>	Use <code>n</code> screen rows (clamped 24–200).
<code>/Cn</code>	Use <code>n</code> screen columns (clamped 80–132).
<code>/R</code>	Open read-only — <code>:w</code> and <code>ZZ</code> refuse to write.
<code>/B</code>	Keep a backup : rename the prior file to <code>name.BAK</code> on save.

Examples:

```
A>VIEDIT REPORT.TXT          edit REPORT.TXT at 80x24 (or auto-detected size)
A>VIEDIT REPORT.TXT /L43     edit on a 43-line terminal
A>VIEDIT BIOS.ASM /R         view a file read-only
A>VIEDIT REPORT.TXT /B       keep the prior version as REPORT.BAK on save
A>VIEDIT                     start with an empty buffer
```

Geometry comes from (in order of precedence) the `/Ln / /Cn` switches, then a `VIEDIT.CFG` file on the current drive (`lines= / columns= lines`), then VT100 auto-detection, then the 80×24 default. A terminal that ignores the size query just falls back to the default after a brief wait.

To save and quit: `:wq` (Return) or `ZZ`. To quit discarding edits: `:q!`.

Command summary

```
MOVE  h j k l  0 ^ $      w b e  W B E
      f F t T  ; ,      gg G   nG
      H M L screen top/mid/bottom
      ^F ^B page  ^D ^U half  %  match bracket

INSERT i a I A   o O open line  R replace mode

EDIT   x del   r{c} replace  ~ case  J join
      d c y + motion  dd yy cc  D C S s
      >> << indent   p P put   u undo   . repeat

FIND   / ? search   n N next/prev  * # word under cursor
SUBST  :s/old/new/[g]  :%s whole file  :.,,$s to end  :N,Ms / : 'a,'b range

MARK   m{a-z} set    `a exact / 'a line jump  y'a d'a c'a range

MISC   ^G status    ^L redraw   K help

FILE   :w write     :w file save-as  :r file read   :e file edit
EXIT   :q quit      :q! force       :wq :x ZZ  write+quit
```

See [VIEDIT.md](#) for full descriptions and examples.

Limitations

- **Large files edit in virtual mode, with restrictions.** A file that fits the TPA (roughly 24 KB on a 56K system) is held fully resident and fast. A larger file opens automatically in **virtual mode**, paging a sliding window to two scratch files (`name. $$$` , `name. $$B`) beside it. Virtual mode is slower (scrolling and searching far do disk I/O), limits **undo** to the current window (`u` becomes a no-op once the window pages), **loses a mark that scrolls out** of the window, makes a save **terminal** (you save and quit — the scratch is consumed), and shows an **approximate** `^G` line total (`>=N`) until the tail pages in. It needs free disk space for the scratch, cleaned up on quit. Only a genuinely enormous file (beyond ~2 GB) is still refused, with *"File too large to edit"*.
- **Single-level undo.** `u` undoes only the most recent change (there is no redo, and no undo history). `.` repeats the last change.
- **Bounded yank/undo range.** A single operation must fit the yank/undo buffer: **character-wise** yank/delete/change up to ~250 bytes; **line-wise** up to ~1000 bytes *and* at most 255 lines — i.e. on the order of a dozen lines of typical 80-column text. An oversize range reports *"Range too large"* and changes nothing rather than truncating.
- **Maximum line length ~255 characters.** Lines are displayed and edited through a 255-cell buffer (`MAXCOLS`); longer content is clamped on display, and `J` (join) refuses with *"Line too long"* rather than build an over-long line.
- **Literal search and substitute — no regular expressions.** `/` , `?` , `n` , `N` , `*` , `#` , and `:s` match plain text only. Search scans the resident buffer.
- **Geometry is clamped** to 24–200 rows and 80–132 columns, and may only enlarge past the 80×24 minimum.
- **One file, one window.** No multiple buffers, splits, or windows; `:e` replaces the current buffer.
- **Requires a VT100-compatible terminal** for screen control (and for size auto-detection).

Building

The editor is one program assembled from several modules and linked into `VIEDIT.COM` . `VICMD` was split by feature so no single source file is much over ~44 KB (small enough to edit on the CP/M box itself) and the single `L80` command still fits CP/M's ~127-char console line.

Source modules (link order):

Module	Role
VI	entry, TPA sizing, main loop, terminal autodetect
VISCREEN	VT100 output, screen redraw
VIKEY	key decoder
VIBUF	gap-buffer text engine
VIWIN	in-RAM line-navigation primitives
VIFILEIO	file load / save
VIVM	virtual buffering (paging text to disk)
VICMD	command dispatch + motions
VICMDOPS	operators <code>d / c / y</code> + insert/replace
VICMDEXC	ex command line (<code>:</code>), <code>:r / :w / :e / :stat</code>
VICMDSCH	find-char, search, fast literal search, <code>%</code>
VICMDUND	undo / repeat / shift / marks
VICMDSUB	ex substitute (<code>:s</code>)
VICMDDAT	shared data, dispatch tables, build stamp
VIEND	end-of-image marker (linked last)

`VI.INC` is the shared `INCLUDE` (constants); `VIBLD.INC` carries the build timestamp.

To rebuild on a CP/M system with M80 and L80 present, run `SUBMIT VIEDIT` : it assembles every module with M80 and links them with L80 into `VIEDIT.COM` .

`VIEDIT.SUB` assumes all fifteen `.MAC` modules, `VI.INC` , and `VIBLD.INC` are on the current drive. `VIBLD.INC` just defines the build-stamp string that `:stat` prints, so if you do not have one, a single line will do:

```
BLDTS:  DB      '1980-01-01 00:00',0
```

`VIEDIT.SUB` also opens with two `R` lines and closes with three `W` lines. Those drive the host-file-transfer utilities of the simulator this editor is developed on; on real hardware the sources are already on the disk, so CP/M reports `R?` / `W?` for each and carries on with the assembly. Trim them if you prefer a quiet build.

The link order matters: **`VIEND` must come last** (it marks the end of the program image), and the whole `L80` command has to fit CP/M's ~127-character console line — which is why `VICMD` is split into as many modules as it is. The current command is 121 characters.